TraceByte - Signature Carving Evaluation

Digital Forensics Project Based Learning | DF-PBL-016 SAP ID: 500119665 **Report version:** 1.0 | **Evaluation run:** `run-20261027T141506Z-489d1acf` **Run timestamp:** 2026-10-27 14:15:06 UTC

1. Project brief and objective

This project implements the assigned Signature Carving Evaluation as a controlled experiment and a local interactive workbench. The experiment measures how a signature-based carver detects and extracts known synthetic file fixtures stored contiguously, in fragments, or with some bytes overwritten. It compares candidate locations with a pre-recorded ground-truth map and compares recovered bytes with the original fixture SHA-256 hashes.

The primary question is: **How do storage condition and file extent boundaries affect signature detection and exact recovery?** A detected header is evidence of a matching byte pattern. It does not by itself establish that the candidate is a complete or valid file. File carving based on content and the challenges of fragmentation are discussed in Garfinkel (2007). NIST's Computer Forensic Tool Testing program provides a broader model for testing forensic tools using specifications, test procedures, criteria, test sets, and hardware [1, 2].

Deliverables implemented

- Eight deterministic synthetic fixtures and a SHA-256 baseline manifest.

- Three controlled conditions and ground-truth maps for each generated run.

- A local carver, measurement layer, and per-run output directories.

- A local web workbench for dataset, integrity, carving, results, evidence intake,

and activity-log workflows.

- This evaluation report, an evidence and reproducibility record, and a viva guide.

2. Scope and safe handling

The measured benchmark uses locally generated synthetic data. Real-evidence intake is a separate workflow: it imports a user-selected file or uncompressed raw byte image into a local copy, hashes that copy, and scans supported signatures. Real scans do not have known ground truth and are not included in benchmark precision, recall, or exact-recovery scores.

The application does not acquire a live disk, interpret E01/EWF containers or file systems, or establish legal admissibility. Use only evidence that the operator is authorized to examine. Preserve source media, work from a verified copy, record tool versions and commands, and independently validate any candidate using an appropriate format parser. The project does not alter the selected source file.

3. Method

3.1 Dataset and ground truth

The baseline contains eight synthetic fixtures: three JPEG/PDF/ZIP signature fixtures without their own delimiters, one signature-free negative control, one container fixture with an embedded JPEG signature, and three delimiter-bearing JPEG/PDF/ZIP fixtures. They contain signature-shaped bytes and deterministic payloads; they are not complete, standards-conformant media files. Thus the evaluation tests detection and exact byte recovery, not successful opening in consumer applications.

The baseline manifest stores fixture names, sizes, hashes, signature metadata, and tool information. Each run stores a run manifest, condition-specific image, ground-truth map, carve results, and metrics. Ground truth is generated before carving and used by the evaluator for scoring. The carver itself scans the image; it does not consult the

ground-truth locations to find candidates.

3.2 Conditions and metrics

The run evaluates `contiguous`, `fragmented`, and `overwritten` images. For each condition, detection counts compare candidate signatures and offsets with the ground-truth signature instances. Precision is TP/(TP+FP), recall is TP/(TP+FN), and F1 is their harmonic mean. Exact recovery requires candidate bytes to match a ground-truth fixture's SHA-256. The exact-match count below is shown separately as true exact fixture matches out of eight fixtures. Mean byte coverage measures the portion of a fixture's bytes represented in the extracted candidate; data completeness measures the portion still present in the condition image. These last two measures are project-specific diagnostics, not a general forensic recovery standard.

3.3 Reproduction

From the repository root in PowerShell:

```
cd D:\TraceByte
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python app.py
```

Open `http://127.0.0.1:8510`, select **Carving lab**, and run the three-condition experiment. The reproducible benchmark and automated checks can also be run with:

```
python -c "from src.experiments import run_all_conditions; print(run_all_conditions())"
python -m pytest -q
```

Each execution receives a new run ID. For the results in this report, select `run-20261027T141506Z-489d1acf` in the Results page or read the three `metrics.json` files beneath `data/experiments/<run_id>/`. Re-running creates a new run and should not be expected to reuse this identifier.

4. Results

Values below were transcribed from the saved run's `metrics.json` files. Detection is scored against seven ground-truth signature instances in each image; exact fixture matches are reported out of the eight baseline fixtures.

Condition	Detection TP/FP/FN	Precision	Recall	F1	Exact SHA-256 matches	Mean byte coverage	Data completeness
Contiguous	7 / 2 / 0	0.7778	1.0000	0.8750	3 / 8 (37.5%)	100.00%	100.00%
Fragmented	7 / 2 / 0	0.7778	1.0000	0.8750	0 / 8 (0.0%)	33.64%	100.00%
Partially overwritten	7 / 2 / 0	0.7778	1.0000	0.8750	0 / 8 (0.0%)	75.15%	75.02%

The per-signature detection counts were JPEG 3 TP/1 FP, PDF 2 TP/0 FP, and ZIP 2 TP/0 FP in each condition. The embedded JPEG signature is detected as a signature, but is not a top-level file recovery in the fixture outcome table.

5. Analysis and failure modes

Detection did not imply recovery. All seven ground-truth signature instances were found in each condition, while two extra candidates per image had no ground-truth file starting at their offsets. Consequently, the observed detection precision was 7/9 (0.7778). The condition generator deliberately plants two decoy signatures per image, while checking the filler for unintended signatures. This is a controlled test of metric behavior, not an estimate of false-positive behavior on arbitrary media.

Contiguous data enabled byte coverage but not always exact recovery. Three of the eight fixture outputs matched their expected SHA-256. Several candidates were overlong or truncated because the extent policy encountered an end marker belonging to a later candidate or reached end of stream. A candidate can contain all expected bytes (100% coverage) and still fail exact hash comparison because it also contains extra bytes. The synthetic fixtures' simplified structures also limit what can be concluded about real JPEG, PDF, or ZIP parsers.

Fragmented data exposed a limitation of linear extraction. Completeness was 100%, meaning the experiment placed all original bytes somewhere in the image, but mean byte coverage was 33.64% and no fixture matched its baseline hash. The linear extraction policy did not reassemble the non-contiguous pieces. This does not mean all real fragmented files have this outcome; filesystem metadata, fragment-aware carving, format validation, and image layout can change results.

Overwriting reduced both completeness and candidate coverage. In this run, data completeness was 75.02% and mean coverage was 75.15%, with no exact matches. The slight difference between these measures reflects their distinct definitions and aggregation over fixtures. The result shows how the implemented overwrite condition affected this dataset; it is not a claim about irrecoverability of particular real-world media.

Alternative explanations and limitations

- Results depend on this eight-fixture synthetic dataset, its placement, the

supported signature set, and the scanner's extent limits and policies.

- Signature recognition covers JPEG, PDF, and ZIP in the measured fixture set.

The separate real-evidence scanner recognizes JPEG, PNG, GIF, PDF, and ZIP, but those scans are not benchmarked here.

- The fixtures are signature-shaped rather than valid full-format files; no

independent format parser was used to validate benchmark outputs.

- Mean byte coverage assumes contiguous correspondence and can understate the

amount of data present in a fragmented layout. It is not equivalent to recovery.

- One saved run provides a reproducible example, not statistical confidence

across randomized placements, multiple tools, or real media.

- The activity log is append-only at application level, but is neither signed nor

hash-chained. The baseline manifest is not digitally signed.

6. Evidence and reproducibility record

The project evidence inventory for the reported run is:

Artifact	Location	Purpose
Baseline fixtures	`data/originals/`	Synthetic known originals
Baseline manifest	`data/originals/manifest.json`	Fixture metadata and SHA-256 baselines
Run manifest	`data/experiments/run-20261027T141506Z-489d1acf/run.json`	Run ID, status, condition records
Condition inputs and labels	`data/experiments/run-20261027T141506Z-489d1acf/exp-*/image.bin`, `ground_truth.json`	Images scanned and pre-generated labels
Carve records and metrics	`data/experiments/run-20261027T141506Z-489d1acf/exp-*/carve_results.json`, `metrics.json`	Candidate output and reported measures

Artifact	Location	Purpose
Recovered candidates	`data/recovered/run-20261027T141506Z-489d1acf/`	Extracted candidate bytes
Provenance events	`logs/experiment_log.jsonl`	Local activity and experiment events

Every image and fixture has a SHA-256 recorded in the corresponding manifest or run output. Recompute the hash of a file with:

```
python -c "import sys; sys.path.insert(0,'.'); from src.hashing import sha256_file;
print(sha256_file(sys.argv[1]))" data\originals\synthetic_photo_01.jpg
```

The software suite contains 170 tests. Run `python -m pytest -q` and preserve its output with the submission. Tests cover the engine and workbench behavior; passing software tests do not validate the accuracy of the experiment's assumptions or make the tool forensically certified.

7. Conclusion

For the saved synthetic run, signature detection found all seven ground-truth signature instances in each condition, but exact recovery varied with layout: 3/8 fixtures matched their baselines when contiguous, and none matched in the fragmented or partially overwritten conditions. The results support the limited conclusion that this project's linear signature and extent policies detect more than they can exactly recover under these controlled conditions. They do not establish performance on real evidence or prove that any real candidate is valid.

8. Sources and assistance disclosure

1. Garfinkel, S. L. (2007). "Carving contiguous and fragmented files with fast

object validation." *Digital Investigation*, 4 (Supplement), S2-S12. https://doi.org/10.1016/j.diin.2007.06.017

2. National Institute of Standards and Technology. "Computer Forensics Tool

Testing Program (CFTT)."
https://www.nist.gov/itl/csd/secure-systems-and-applications/computer-forensics-tool-testing-program-cftt

3. Digital Forensics PBL assignment brief supplied for DF-PBL-016. Requirements

were used to map the project scope and submission checklist.

AI assistance was used to review project files and revise documentation for clarity and assignment alignment. The student should verify every reported value against the saved JSON artifacts, run the project independently, understand the analysis, and disclose assistance in the format required by the course. No student name is asserted here; the supplied SAP ID is used as the identifier.